R2.07 TP 4

Recherche de plus courte chaîne

4 mars 2026 – B. Colombel

Votre compte-rendu est à rendre sous la forme d'un notebook jupyter sous le nom

`R2.07-TP4-NOM-Prenom.ipynb`

dans le dossier prévu à cet effet sur Ametice.

Sommaire

1 Rappels sur les graphes pondérés **1**

2 Algorithme de Bellman-Ford (simplifié) **3**

3 Algorithme de Dijkstra **4**

---

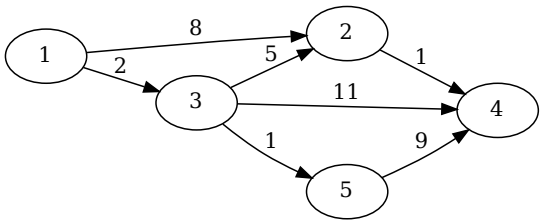
1 Rappels sur les graphes pondérés

Un graphe *pondéré* est un graphe auquel on ajoute un poids sur ses arcs ou ses arêtes.
Nous avons vu en cours qu'un graphe pondéré peut être représenté par sa matrice de poids M définie de la façon suivante :

$$m_{ij} = \begin{cases} +\infty \text{ s'il n'y a pas d'arête/arc entre les sommets } i \text{ et } j \\ \text{valuation de l'arc/arête } \{i,j\} \end{cases}$$

Par exemple, la matrice

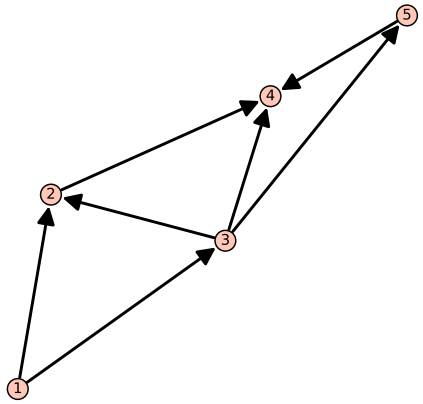
$$M = \begin{pmatrix} +\infty & 8 & 2 & +\infty & +\infty \\ +\infty & +\infty & +\infty & 1 & +\infty \\ +\infty & 5 & +\infty & 11 & 1 \\ +\infty & +\infty & +\infty & +\infty & +\infty \\ +\infty & +\infty & +\infty & 9 & +\infty \end{pmatrix}$$



est la matrice de poids du graphe ci-contre.

Pour créer un graphe pondéré avec *sagemath*, on n'utilise pas directement la matrice des poids :

[1]:
```
g = DiGraph(weighted = True)
g.add_edges([(1, 2, 8), (1,3,2), (2,4,1), (3,2,5), (3,4, 11), (3,5,1), (5, 4, 9)])
g.show()
```



On peut retrouver la matrice des poids mais **attention**, les $+\infty$ sont signifiés par des 0 !

[2] :
```
M = g.weighted_adjacency_matrix()
print(M)
```

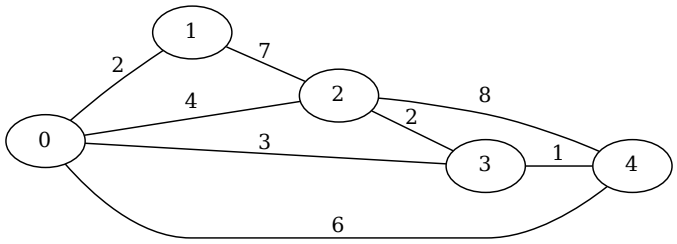
[2] :
```
[ 0  8  2  0  0]
[ 0  0  0  1  0]
[ 0  5  0 11  1]
[ 0  0  0  0  0]
[ 0  0  0  9  0]
```

Les arcs, sont alors des triplets (i,j,p) où le 3e élément correspond au poids de l'arc (i,j).

[3] :
```
g.edges()
```

[3] :
```
[(1, 2, 8), (1, 3, 2), (2, 4, 1), (3, 2, 5), (3, 4, 11), (3, 5, 1), (5, 4, 9)]
```

Vous testerez vos algorithmes avec le graphe `g` défini précédemment et le graphe `g2` :



[4] :
```
g2 = Graph()
g2.add_edges([(0,1,2), (0,2,4), (0,3,3), (0,4,6), (1,2,7), (2,3,2), (2,4,8), (3,4,1)])
M2 = g2.weighted_adjacency_matrix()
print(M2)
```

[4] :
```
[0 2 4 3 6]
[2 0 7 4 0]
[4 7 0 2 8]
[3 4 2 0 0]
[6 0 8 0 0]
```

Remarques. 1. *sagemath* possède une représentation de l'infini : `Infinity`.

2. `Infinity` vérifie les propriétés classiques :

[7]:
```
print(Infinity > 9999)
print(Infinity + 82 == Infinity)
print(Infinity + 1 > Infinity)
print(Infinity*(-72) == -Infinity)
```

[7]:
```
True
True
False
True
```

2 Algorithme de Bellman-Ford (simplifié)

Le but de cette partie est d'implémenter l'algorithme de Bellman-Ford qui permet de déterminer, sous conditions, un plus court chemin entre un sommet choisi et tous les autres sommets du graphe.
On rappelle l'algorithme de Belleman-Ford dans une version simplifiée (qui détermine les plus courtes distances sans fournir le chemin) :

Algorithme de Bellman-Ford (simplifié)

Entrées : matrice de poids M d'un graphe G
Initialisation : `Dist` = liste des distances depuis le sommet 0, initialisées à l'infini ;
`Dist`$[0] = 0$;

```
1  début
2      tant que Dist change faire
3          pour tout sommet i faire
4              pour tout sommet j faire
5                  si Dist[i] + M[i][j] < Dist[j] alors
6                      Dist[j] = Dist[i] + M[i][j]
7                  fin
8              fin
9          fin
10     fin
11 fin
```

Sorties : Retourner `Dist`

Exercice 1. Construire une fonction `Bellman_Ford(M)`, qui renvoie les distances du plus court chemin du sommet 0 à tous les autres, dans un graphe orienté (ou non) de matrice de poids M, à l'aide de l'algorithme de Bellman-Ford.

Exemples d'utilisation.

[7]:
```
Bellman_Ford(M)
```

[7]:
```
[0, 7, 2, 8, 3]
```

[8]:
```
Bellman_Ford(M2)
```

[8]:
```
[0, 2, 4, 3, 4]
```

3 Algorithme de Dijkstra

Algorithme de Dijkstra (simplifié)

Entrées : matrice de poids M d'un graphe g
Initialisation : `Dist` = liste des distances depuis le 1er sommet, initialisées à l'infini ;
`Dist`$[0] = 0$;

```
1  début
2      tant que des sommets ne sont pas marqués faire
3          choisir un sommet i non marqué, tel que Dist[i] soit minimale ;
4          marquer i ;
5          pour tout sommet j non marqué faire
6              si Dist[i] + M[i][j] < Dist[j] alors
7                  Dist[j] = Dist[i] + M[i][j] ;
8              fin
9          fin
10     fin
11 fin
```

Sorties : Retourner `Dist`

Exercice 2. Construire une fonction `dijkstra(M)`, qui renvoie les distances du plus court chemin du sommet 1 à tous les autres, dans un graphe de matrice de poids M, à l'aide de l'algorithme de Dijkstra.

Tester votre fonction avec les graphes `g` et `g2`.

Remarque. On pourra par exemple créer une liste `marquage`, avec `marquage(i) = 1` si le sommet `i` est marqué, `0` sinon.

Si vous avez le temps, améliorez votre code pour qu'il affiche en sortie le chemin le plus court.